Learning Theory

Overfitting, Regularization, Validation

Overfitting (and Underfitting)

Supervised Learning

- Our Strategy:
  - Ensure E_{in} and E_{out} are close.
  - Train to minimize E_{in}.

- Ensure E_{in} and E_{out} are close:
  - Choose simple model.
  - Leads to underfitting.

- Train to minimize E_{in}:
  - Choose complex model.
  - Leads to overfitting.



Overfitting

- Why does overfitting occur?
  - Symptom: E_{in} becomes small, but E_{out} is large.
  - $\Pr[|E_{in}(h^*) - E_{out}(h^*)| > \varepsilon] < 8((2n)^d+1)e^{-\varepsilon^2 n/8}$

- VC-bound is just a guarantee, not in itself an explanation.

- An explanation:
  - Powerful model uses its degrees of freedom to fit noise in the data.
  - Let's have a closer look.

Overfitting

- Case Study: Polynomials

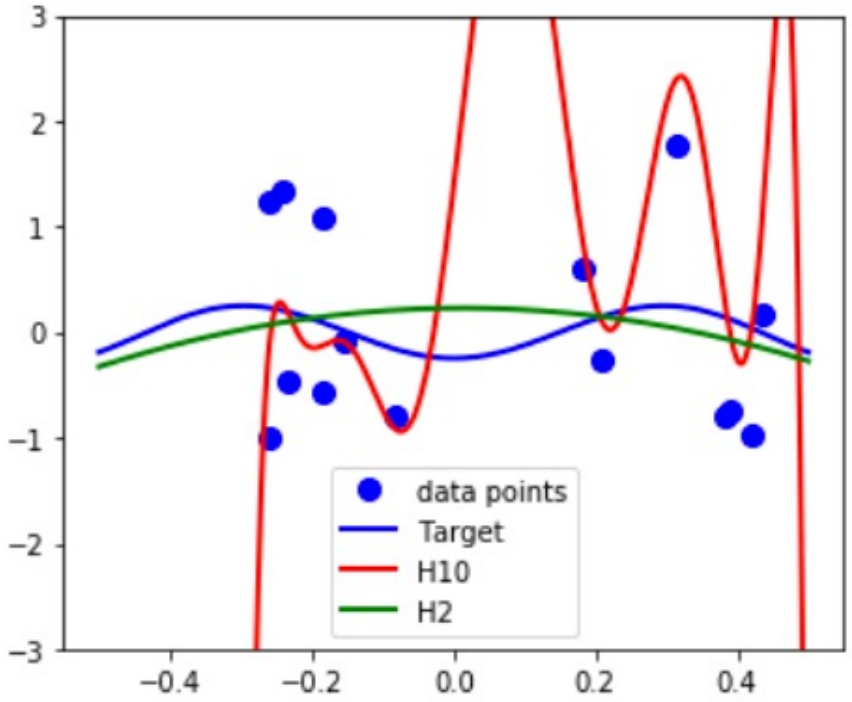
- Unknown target polynomial f(x). Assume x in [-½, ½].
- Learning algorithm:
  - Do feature transform $x \rightarrow (1,x,x^2,\ldots,x^k)$
  - Least squared fit (linear regression).
    - $w^T(1,x,x^2,\ldots,x^k) = w_0+w_1x+w_2x^2+\ldots+w_kx^k$

- Data: 15 random samples.

- Hypothesis sets:
  - H_2 = {degree 2 polynomials} //k=2, 3 params
  - H_{10} = {degree 10 polynomials} //k=10, 11 params

Overfitting

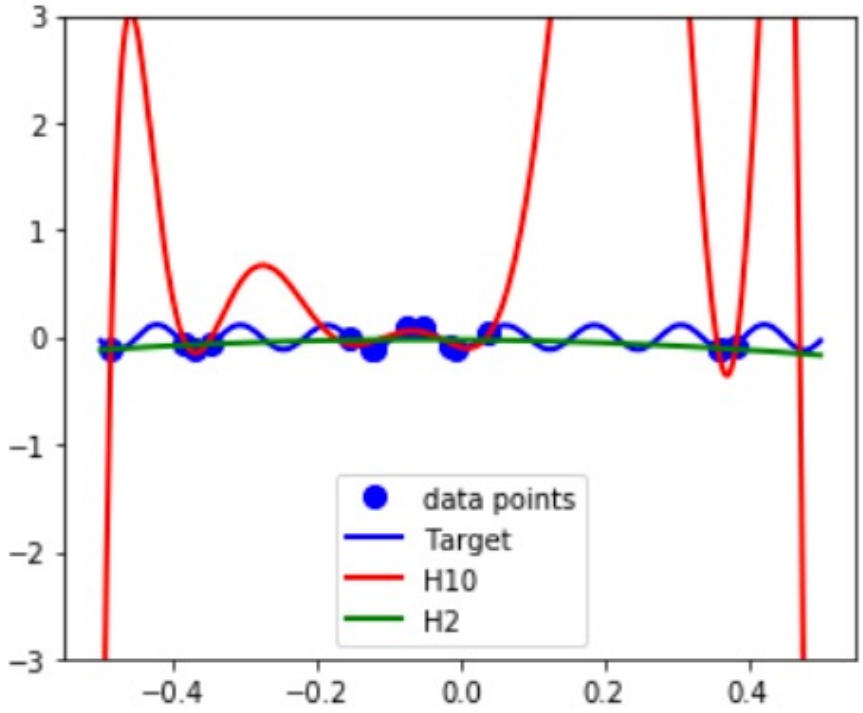
- Case Study: Polynomials
- Hypothesis sets:
  - H_2 = {degree 2 polynomials}
  - H_{10} = {degree 10 polynomials}
- Target Function:
  - $f(x) = \sum_{i=0}^{10} a_i x^i + \sigma$
- Degree 10 with noise.



Notebook Experiments

Overfitting

- Case Study: Polynomials
- Hypothesis sets:
  - H_2 = {degree 2 polynomials}
  - H_{10} = {degree 10 polynomials}
- Target Function:
  - $f(x) = \sum_{i=0}^{50} a_i x^i$
- Degree 50 **without** noise.

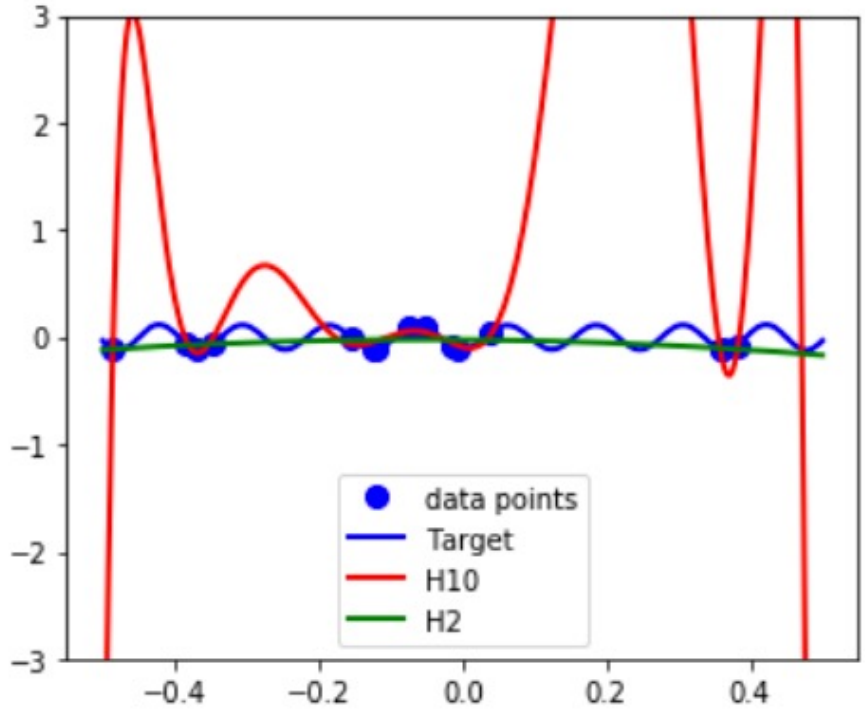


Notebook Experiments

Overfitting

- Why?
- Deterministic noise:
- For any hypothesis set H, there is a best fit h* to f in H. What cannot be fit, is a form of noise for H.
- With complex model, will attempt to fit this ”residual”.

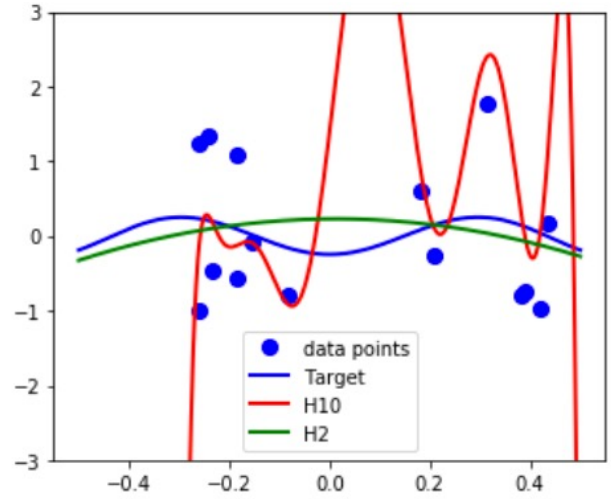
- ”Classic” noise called: Stochastic noise.



Overfitting

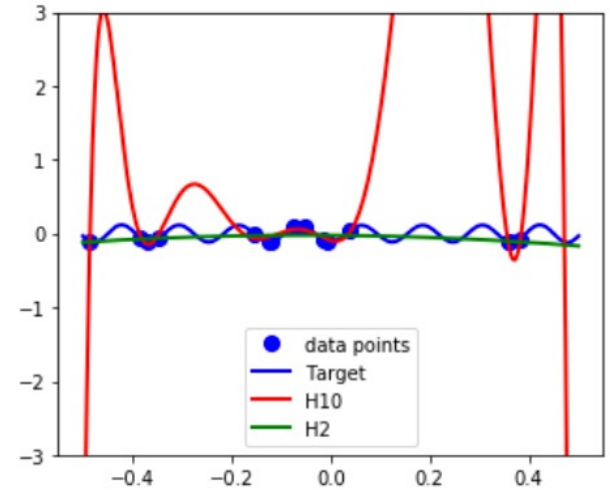
- Does overfitting always occur?
- Always choose simple model?

- No!
- Given 50 data points and hypotheses H_{50}, would perfectly fit second example.



- Overfitting occurs when use too complex model compared to amount of data available.

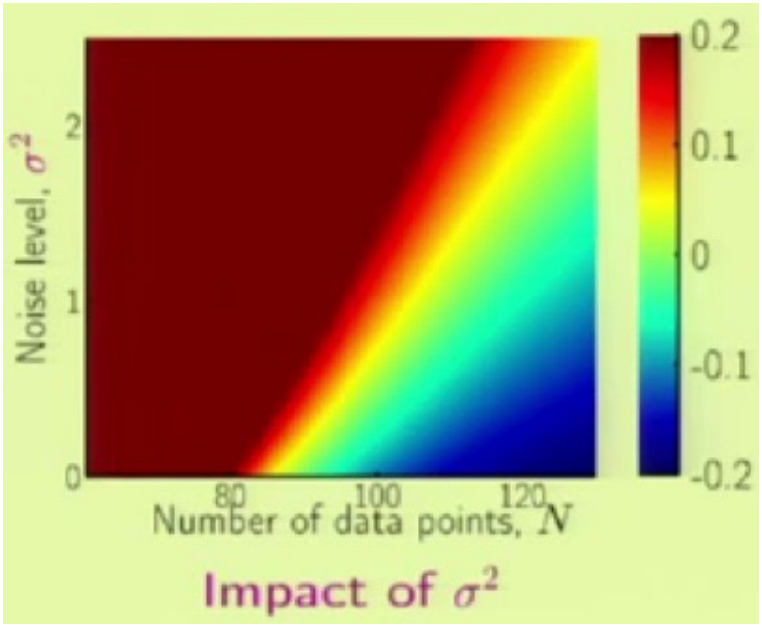
- More data allows more complex model.

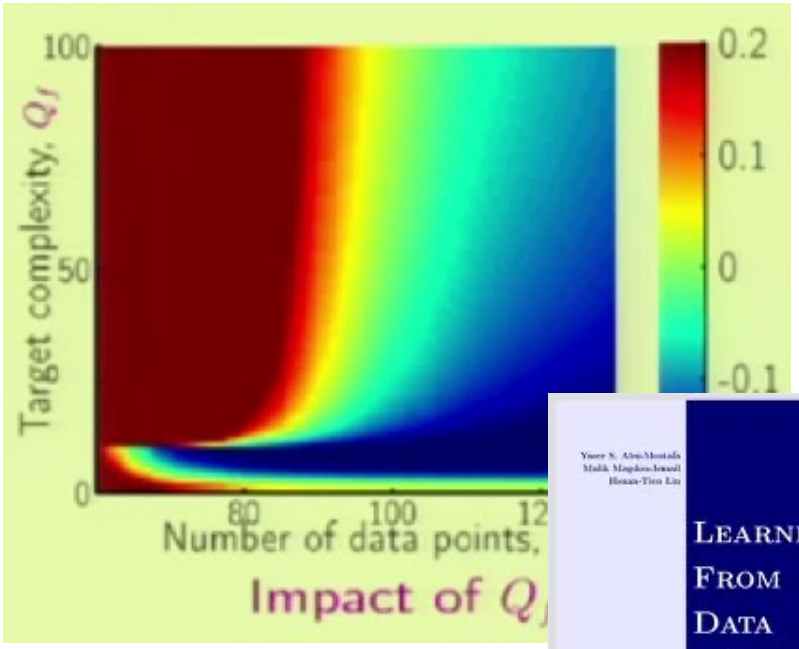


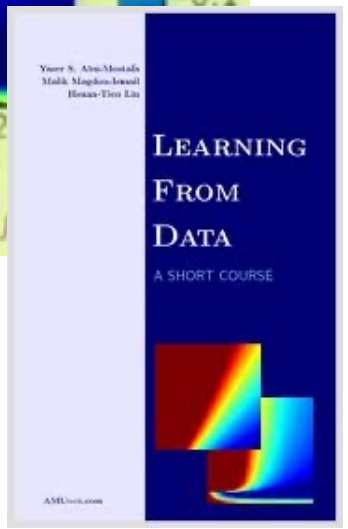
Notebook Experiments

Overfitting

- H_2 vs H_{10}. Plot $E_{out}(H_{10})$-$E_{out}(H_2)$.







- Data ↑ Overfitting ↓
- Noise ↑ Overfitting ↑
- Target Complexity ↑ Overfitting ↑

Learning Theory

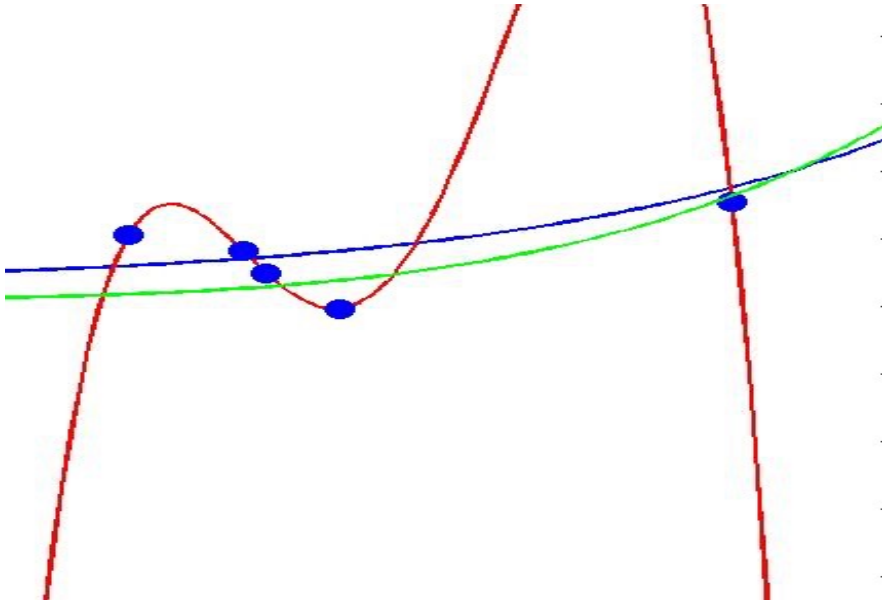
Overfitting, Regularization, Validation

Regularization

Combating overfitting

Our Dilemma

- Want to use complex hypothesis set to avoid underfitting.
- We do not want to overfit.



- Green and Red are same order polynomials (same H).
- Red minimized E_{in} to perfection. Green has much lower E_{out}.
- Red mislead by noise (more than Green).
- Can we ensure that we learn the Green?

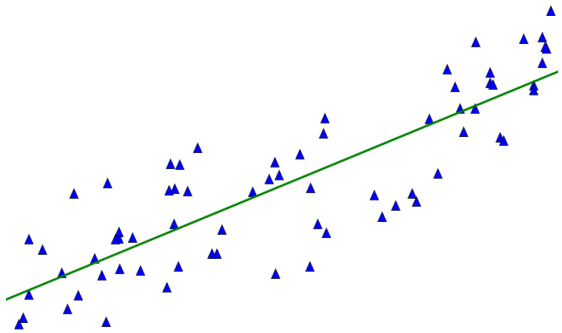
Regularization

- Idea:
  - Use complex hypothesis set.
  - Prevent learning algorithm from fitting noise.
- How:
  - Penalize learning algorithm by making complex hypotheses more expensive to use.
  - Will use complex hypothesis only if decreases E_{in} drastically.

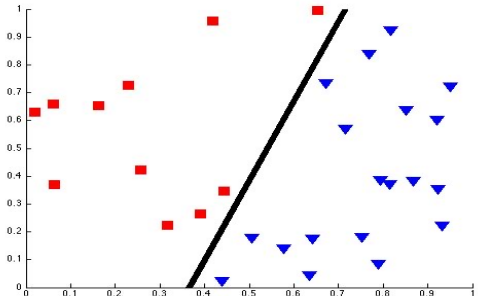
- Inspired by VC-bound:
  - $E_{out}(h) \leq E_{in}(h) + \Omega(H)$ ← Term that grows with complexity of H.
- Instead:
  - $E_{out}(h) \leq E_{in}(h) + \Omega(h)$ ← Term that grows with complexity of h.

Regularization

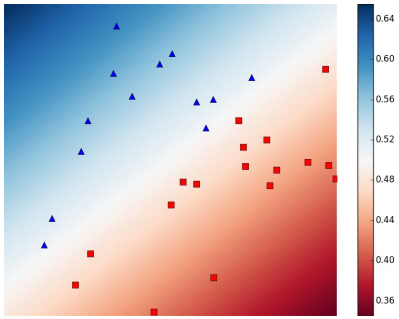
- What is a simple hypothesis?



$h(x) = w^\intercal x$



$h(x) = \text{sign}(w^\intercal x)$

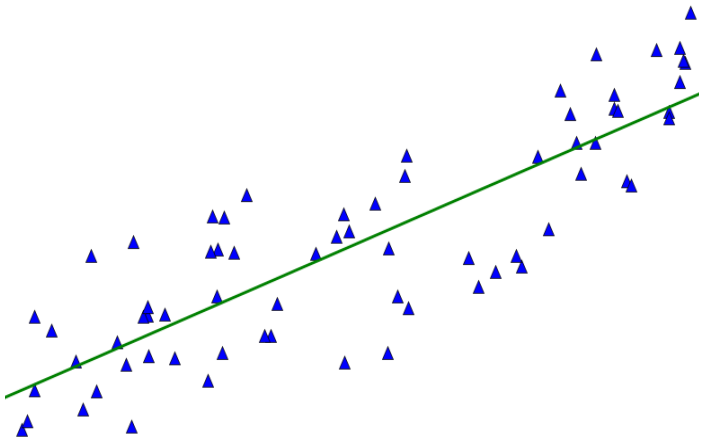


$h(x) = \sigma(w^\intercal x)$

- $h_1(x)=24 + 939x^1 - 1039x^2 + 667x^3 + 918x^4$
- $h_2(x)=4 + 3x^1 - 2x^2 + 7x^3 + 10x^4$
- Which is simpler?
  - h_2 less sensitive to noise / small changes in input.

Regularization

- Weight Decay
- Define simpler in terms of weights:
  - $\Omega(h(w)) = \lambda\|w\|^2$
  - $\lambda > 0$ a parameter.



$$h(x) = w^\intercal x$$

- Recall $\|w\|^2 = \sum_i w_i^2$

- During training:
  - Minimize $E_{in}(h(w)) + \lambda\|w\|^2$

- Example linear regression:
  - $E_{in}(h(w)) = \sum_{i=1}^{n}(w^T x_i - y_i)^2$

Regularization

- Linear Regression with Weight Decay
  - Minimize
    - $\frac{1}{n}\sum_{i=1}^{n}(w^T x_i - y_i)^2 + \lambda\|w\|^2$

$\lambda\|w\|^2 = \lambda w^T w$

Matrix of derivatives: $2\lambda w^T$

Gradient is transpose.

- Compute gradient:
  - $\nabla_w = \frac{1}{n}(2X^T Xw - 2X^T y) + 2\lambda w$
- Set to 0:
  - Normal (λ=0):
    - $w = (X^T X)^{-1} X^T y$
  - With Weight Decay:
    - $w = (X^T X + n\lambda I)^{-1} X^T y$

Can assume $X^T X$ invertible by previous arguments.

Still invertible?

Yes!

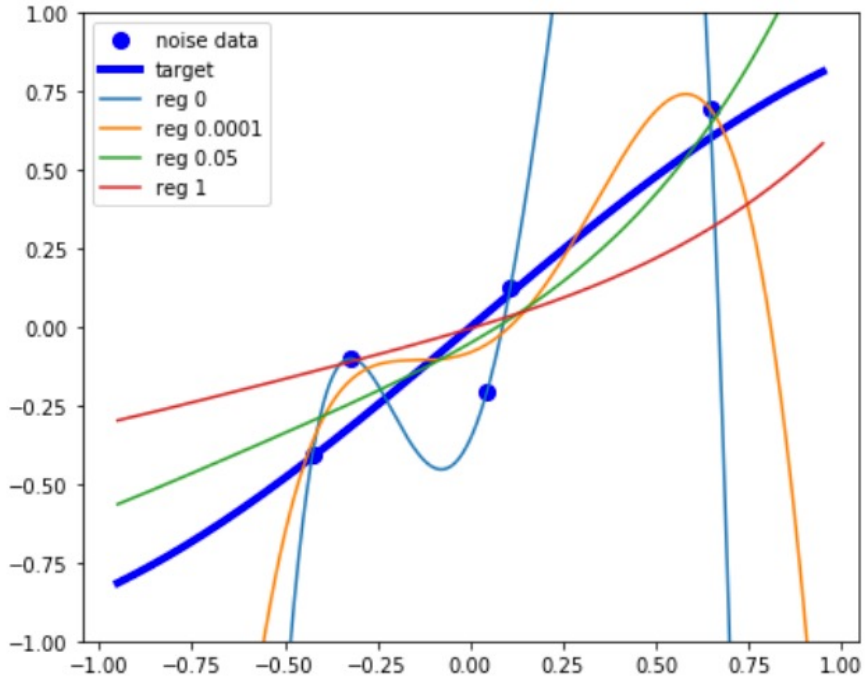
$X^T X$ has all eigens >0.

$n\lambda I$ adds λ to all eigens.

Regularization

- Linear Regression with Weight Decay
  - Minimize
    - $\frac{1}{n}\sum_{i=1}^{n}(w^T x_i - y_i)^2 + \lambda\|w\|^2$
- Does regularization help?
  - For linear models,
    it often does.
  - Depends a lot on model.

- For linear regression, even has its own name:
  Ridge Regression



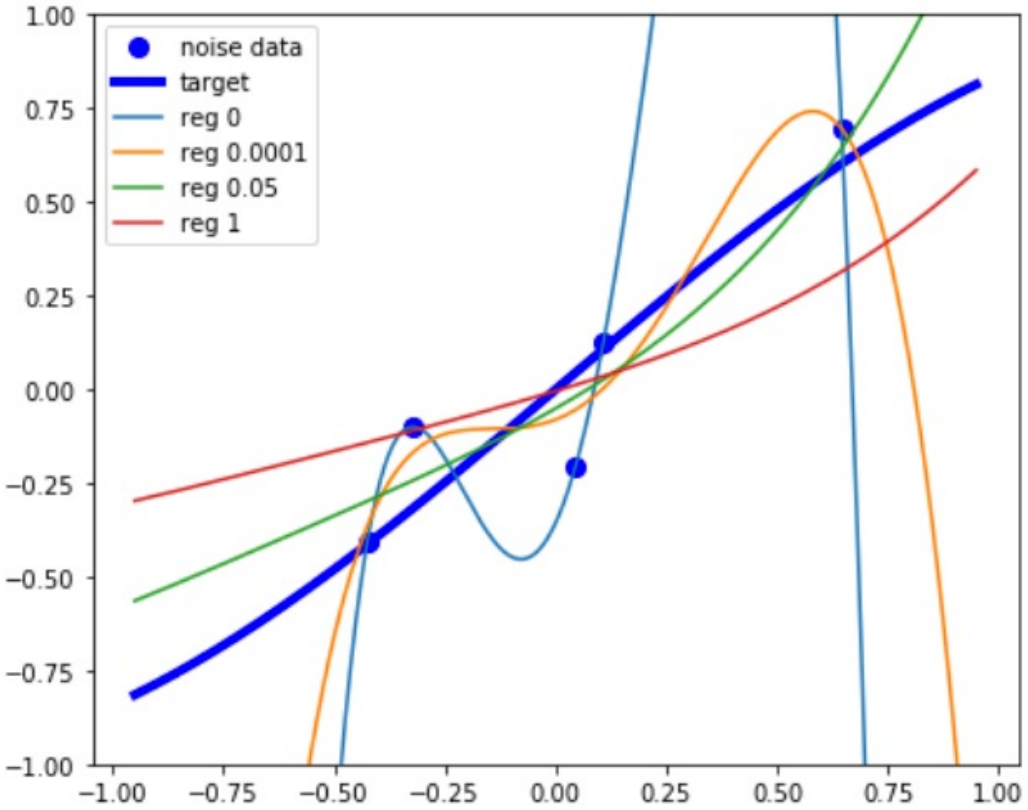
Notebook Experiments

Regularization

- Summary:
  - Regularization is any modification made to learning algorithm with the goal of reducing E_{out} at the cost of increasing E_{in}.
  - It is a tool for making the learning algorithm prefer some hypotheses over others (so choose wisely).
    - Unpreferable solutions chosen only if give large gains in E_{in}.
  - More an art than a science.
  - Weight decay a universal technique.
  - Many other regularizers exists.
  - Extremely important. Necessary evil.

Regularization

- But wait, how do we know which λ to use??



Learning Theory

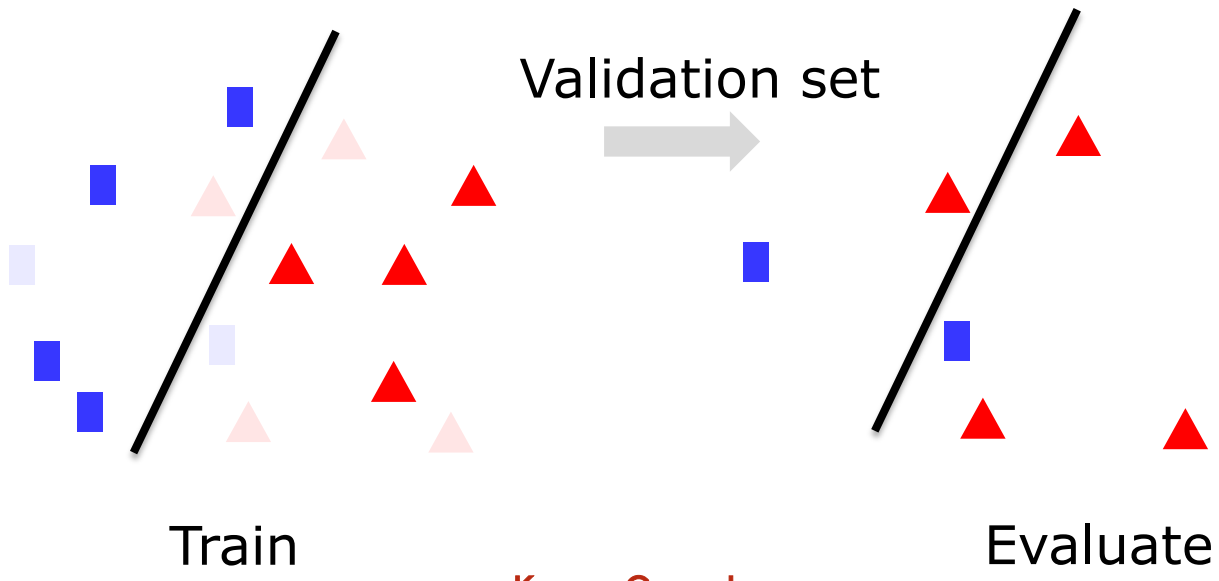
Overfitting, Regularization, Validation

Validation

Validation vs Regularization

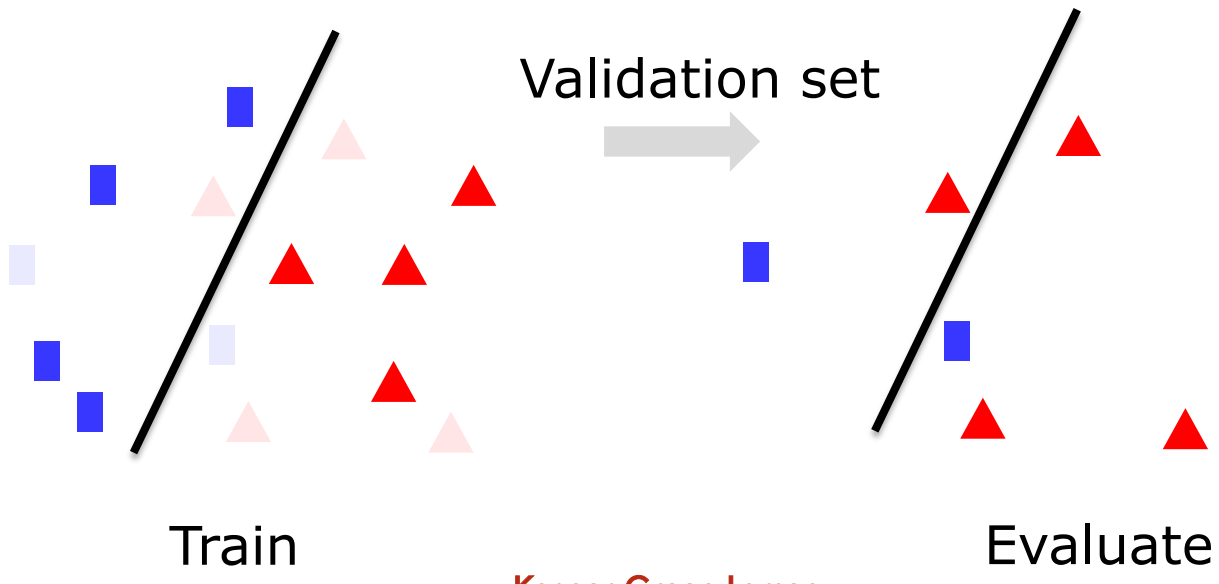
- From learning theory:
  - $E_{out} \leq E_{in} + \Omega(h)$

- Regularization minimized E_{out} by minimizing a proxy of $E_{in} + \Omega(h)$.
- Validation approximates E_{out} directly.



Validation

- Validation template:
  - Set aside k input points for validation (validation set D_{val}).
  - Train on remaining n-k (training set D_{train}).
  - Evaluate hypothesis on D_{val} to get estimate $E_{val} \approx E_{out}$.
  - Learning algorithm makes decisions based on outcome.



Model Selection

- Concrete Example, Model Selection:
  - Need to choose regularization (hyper-)parameter λ.
  - How: Run learning algorithm with many different λ's. All train on D_{train}.
  - Compute $E_{val}(h_1),\ldots,E_{val}(h_M)$, where $h_1,\ldots,h_M$ are hypotheses returned for the M different choices of λ.
  - Return $h^* = h_i$ with the lowest $E_{val}(h_i)$.

- Need:
  - The $E_{val}(h_i)$ must be pretty good estimates of $E_{out}(h_i)$.
  - k must be reasonably large.
- Hoeffding + union bound:
  - $\Pr[\exists i : |E_{val}(h_i) - E_{out}(h_i)| > \varepsilon] < 2Me^{-2\varepsilon^2 k}$

Model Selection

- Concrete Example, Model Selection:
  - Need to choose regularization (hyper-)parameter λ.
  - $\Pr[\exists i : |E_{val}(h_i) - E_{out}(h_i)| > \varepsilon] < 2Me^{-2\varepsilon^2 k}$

- Larger k gives better approximation of E_{out}.
- Larger k gives smaller n-k, i.e. worse E_{out}/E_{val}.

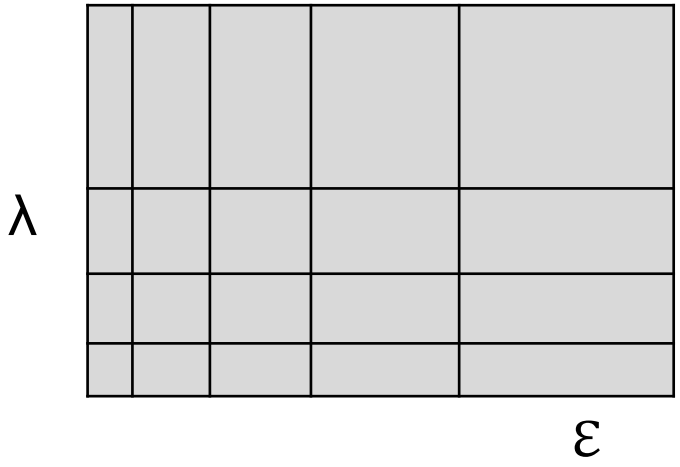
- A Cheat/Trick:
  - Once validation has chosen λ, retrain with that λ and all of data, i.e. $D=D_{train}+D_{val}$.
  - More data should give better fit!

Model Selection

- Hyperparameters:
  - Many learning algorithms have hyperparameters.
    - Parameters not learned by the algorithm, but set by the user.
  - Examples:
    - Regularization parameter λ in ridge regression.
    - Depth of decision trees.
    - #leaves in decision trees.
    - Later: Penalty factor C for Supper Vector Machines.

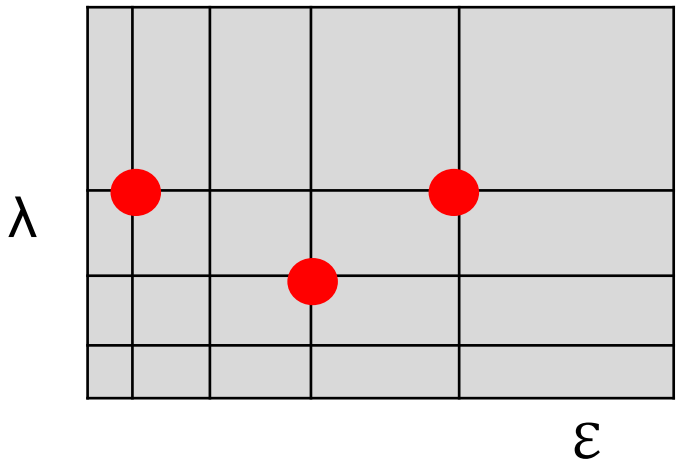
Model Selection

- Hyperparameters:
  - Many learning algorithms have hyperparameters.
- Grid search:
  - Try exponentially changing values for each parameter
    - E.g. λ in 0.3^i for i=0,…,10 gives {1, 0.3, 0.09, …}.
  - Run validation on all choices of parameters.
  - Select parameters that give best E_{val}.



Model Selection

- Hyperparameters:
  - Many learning algorithms have hyperparameters.
- Grid search:
  - #settings grows exponentially with #parameters.
  - If too much: Try random parameters for as long as you can wait.
  - Use parameters with best E_{val} score.



Rule of Thumb

- Reasonable practical choice:



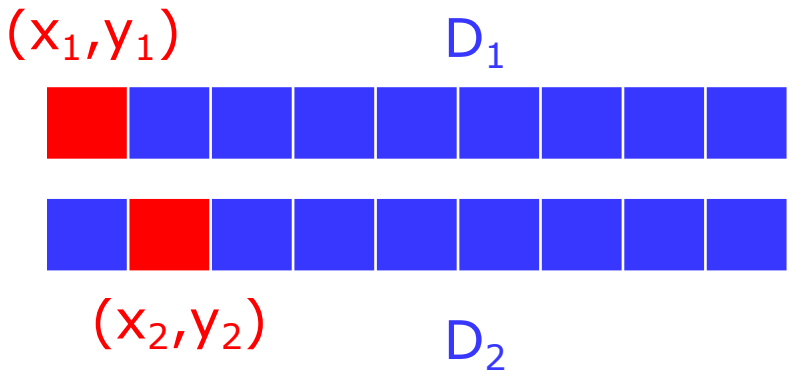
Cross Validation

- Dilemma with k:
  - $\Pr[|E_{val}(h) - E_{out}(h)| > \varepsilon] < 2e^{-2\varepsilon^2 k}$

- Larger k gives better approximation of E_{out}.
- Larger k gives smaller n-k, i.e. worse E_{out}/E_{val}.

- Cross Validation:
  - Try to give better estimates while training on most data.
  - Works well in practice.
  - Hard to give theoretical guarantees.

Cross Validation



- Cross validation with k=1:
  - For each x_i of data items $x_1,\ldots,x_n$:
    - Create $D_i=\{x_1,\ldots,x_n\}/\{x_i\}$.
    - Train on D_i.
    - Compute validation loss e_i on (x_i,y_i).
  - Set $E_{cval} = (1/n)(e_1+e_2+\ldots+e_n)$.

- Model Selection:
  - Do above for every choice of (hyper-)parameters.
- Observations:
  - For each i, use almost all data. Great.
  - For each i, validate on only one point. Bad.
  - Hope: Averaging gives good estimate.

Cannot prove. Not always true.

Cross Validation

- In practice, split in 5.
  - Don’t have time to train n models.

